# The OperationsResearch2 Spring-2025 problem

## Summary

We built an optimization model to assign ships to berths while minimizing costs from waiting time and extra capacity. The model respects berth limits, unloading durations, and ship priorities. We also created visual and text-based reports from the solution for clear scheduling analysis.

## Pyomo formulation

We begin by importing the Pyomo package and creating a model abstract object:

```
from pyomo.environ import *
infinity = float('inf')
model = AbstractModel(name='OR2 project')
```

### Sets

The sets $Ships$ and $Berths$ are declared abstractly using the `Set` component:

```
model.Ships = Set()
model.Berths = Set()
```

### Parameters

Similarly, we add parameters, the model parameters are defined abstractly using the `Param` component:

```
model.M = 999999999
epsilon = 1e-9
model.max_waiting_without_priority = 80

model.berthNumberOfAnchs =
Param(model.Berths,within=NonNegativeIntegers)
model.shipLength = Param(model.Ships,
within=NonNegativeReals,default=infinity)
model.berthMaxCapacityIncrease = Param(model.Berths,
within=NonNegativeReals)
model.berthLag = Param(model.Berths, within=NonNegativeReals)
model.berthCapacity = Param(model.Berths, within=NonNegativeIntegers)
model.berthLength = Param(model.Berths,within=NonNegativeReals)
model.shipArrivalTime = Param(model.Ships, within=NonNegativeReals)
model.unloadShipInAnchTime =
Param(model.Ships,model.Berths,within=NonNegativeReals)
```

```
model.berthIncreaseCapCost =
Param(model.Berths,within=NonNegativeReals)
model.shipCargoSize = Param(model.Ships,within= NonNegativeIntegers)
model.shipPriority= Param(model.Ships, within= NonNegativeReals)
model.shipPriorityMatrix = Param(model.Ships,model.Ships,within =
Binary)
#model.shipLayoverCost = Param(model.Ships, model.Berths,
within=NonNegativeReals)
```

## Variables

The `within` option here is used in these parameter declarations to define expected properties of the parameters. This information is used to perform error checks on the data that is used to initialize the parameter components.

The `Var` component is used to define the decision variables: the `binary` is `{0,1}` to be clear.

```
model.berthCapIncrease = Var(model.Berths, within=NonNegativeIntegers)
model.shipWaitingTime = Var(model.Ships, within=NonNegativeReals)
model.shipWaitingTime_parts =
Var([1,2,3],model.Ships,within=NonNegativeReals)
model.shipWaitingTime_binary_helpers =
Var([1,2],model.Ships,within=Binary)
model.shipCostOfWaiting = Var(model.Ships,within=NonNegativeReals)
model.shipAssignment = Var(model.Ships,model.Berths,within=Binary)
model.shipUnloadStart = Var(model.Ships, within= NonNegativeReals)
model.shipiUnloadBeforekInBerth =
Var(model.Ships,model.Ships,model.Berths,within=Binary)
```

## Constraints

- sequencing constraints.

```
def sequencing_rule1(model, i, k, j):
    if i == k:
        return Constraint.Skip
    return model.shipUnloadStart[k] >= model.shipUnloadStart[i] +
model.unloadShipInAnchTime[i, j] +\
           model.berthLag[j] - model.M * (1 -
model.shipiUnloadBeforekInBerth[i, k, j])
model.Sequencing1 = Constraint(model.Ships, model.Ships, model.Berths,
rule=sequencing_rule1)

def sequencing_rule2(model, i, k, j):
    if i == k:
        return Constraint.Skip
    return model.shipUnloadStart[i] >= model.shipUnloadStart[k] +
model.unloadShipInAnchTime[k, j] +\
           model.berthLag[j] - model.M *
model.shipiUnloadBeforekInBerth[i, k, j]
```

```
#model.Sequencing2 = Constraint(model.Ships, model.Ships,
model.Berths, rule=sequencing_rule2)

def z_activation1(model, i, k, j):
    if i == k:
        return Constraint.Skip
    return model.shipiUnloadBeforekInBerth[i, k, j] <=
model.shipAssignment[i, j]
model.ZActive1 = Constraint(model.Ships, model.Ships, model.Berths,
rule=z_activation1)

def z_activation2(model, i, k, j):
    if i == k:
        return Constraint.Skip
    return model.shipiUnloadBeforekInBerth[i, k, j] <=
model.shipAssignment[k, j]
model.ZActive2 = Constraint(model.Ships, model.Ships, model.Berths,
rule=z_activation2)

def z_activation3(model, i, k, j):
    if i == k:
        return Constraint.Skip
    return model.shipiUnloadBeforekInBerth[i, k, j]
+model.shipiUnloadBeforekInBerth[k, i, j] >=\
          model.shipAssignment[k, j] + model.shipAssignment[i, j] - 1
model.ZActive3 = Constraint(model.Ships, model.Ships, model.Berths,
rule=z_activation3)

def symmetry_rule(model, i, k, j):
    if i >= k:
        return Constraint.Skip
    return model.shipiUnloadBeforekInBerth[i, k, j] +
model.shipiUnloadBeforekInBerth[k, i, j] <= 1
model.ZSymmetry = Constraint(model.Ships, model.Ships, model.Berths,
rule=symmetry_rule)
```

- ordering with priority

```
def priority_seq_rule(model, i, k, d):
    if i != k:
        return model.shipiUnloadBeforekInBerth[i, k, d] >= -
2+model.shipPriorityMatrix[i, k]+model.shipAssignment[i,d]+\

model.shipAssignment[k,d]
    return Constraint.Skip

model.PrioritySequencing = Constraint(model.Ships, model.Ships,
model.Berths, rule=priority_seq_rule)
```

- each ship is assigned to one berth

```
def ship_assignment_rule_top(model,i):
    return sum(model.shipAssignment[i,j] for j in model.Berths) <= 1
def ship_assignment_rule_bottom(model,i):
    return sum(model.shipAssignment[i,j] for j in model.Berths) >= 1
model.ship_assignment_limit_top =
Constraint(model.Ships,rule=ship_assignment_rule_top)
model.ship_assignment_limit_bottom =
Constraint(model.Ships,rule=ship_assignment_rule_bottom)
```

- rules for cost of waiting

```
def cost_sum_rule_bottom(model,i):
    return model.shipCostOfWaiting[i] >=
25*model.shipWaitingTime_parts[1,i]+\

10*model.shipWaitingTime_parts[2,i]+\

5*model.shipWaitingTime_parts[3,i]
model.cost_sum_limit_bottom =
Constraint(model.Ships,rule=cost_sum_rule_bottom)

def cost_sum_rule_top(model,i):
    return model.shipCostOfWaiting[i] <=
25*model.shipWaitingTime_parts[1,i]+\

10*model.shipWaitingTime_parts[2,i]+\

5*model.shipWaitingTime_parts[3,i]
model.cost_sum_limit_top =
Constraint(model.Ships,rule=cost_sum_rule_top)

def waiting_sum_rule_bottom(model,i):
    return model.shipWaitingTime[i] >=
model.shipWaitingTime_parts[1,i]+\

model.shipWaitingTime_parts[2,i]+\

model.shipWaitingTime_parts[3,i]
model.waiting_sum_limit_bottom =
Constraint(model.Ships,rule=waiting_sum_rule_bottom)

def waiting_sum_rule_top(model,i):
    return model.shipWaitingTime[i] <=
model.shipWaitingTime_parts[1,i]+\

model.shipWaitingTime_parts[2,i]+\

model.shipWaitingTime_parts[3,i]
model.waiting_sum_limit_top =
Constraint(model.Ships,rule=waiting_sum_rule_top)
```

```
def cost_part1_bottom(model,i):
    return 10*model.shipWaitingTime_binary_helpers[1,i] <=
model.shipWaitingTime_parts[1,i]
model.cost_part1_bottom_limit =
Constraint(model.Ships,rule=cost_part1_bottom)

def cost_part1_top(model,i):
    return 10 >= model.shipWaitingTime_parts[1,i]
model.cost_part1_top_limit =
Constraint(model.Ships,rule=cost_part1_top)

def cost_part2_bottom(model,i):
    return 30*model.shipWaitingTime_binary_helpers[2,i] <=
model.shipWaitingTime_parts[2,i]
model.cost_part2_bottom_limit =
Constraint(model.Ships,rule=cost_part2_bottom)

def cost_part2_top(model,i):
    return 30*model.shipWaitingTime_binary_helpers[1,i] >=
model.shipWaitingTime_parts[2,i]
model.cost_part2_top_limit =
Constraint(model.Ships,rule=cost_part2_top)

def cost_part3_top(model,i):
    return 30*model.shipWaitingTime_binary_helpers[2,i] >=
model.shipWaitingTime_parts[3,i]
model.cost_part3_top_limit =
Constraint(model.Ships,rule=cost_part3_top)

def max_waiting_rule(model,i):
    return model.shipWaitingTime[i] <= (1-
model.shipPriority[i])*model.max_waiting_without_priority
model.max_waiting_limit =
Constraint(model.Ships,rule=max_waiting_rule)
```

- maximum increase

```
def cap_increase_rule(model,j):
    return model.berthCapIncrease[j] <=
model.berthMaxCapacityIncrease[j]
model.cap_increase_limit = Constraint(model.Berths,
rule=cap_increase_rule)
```

- berth limit of unloaded containers

```
def berth_unload_rule(model, j):
    return sum(model.shipAssignment[i,j]*model.shipCargoSize[i] for i
in model.Ships) <= \
           model.berthCapacity[j]+model.berthCapIncrease[j]
model.berth_unload_limit =
Constraint(model.Berths,rule=berth_unload_rule)
```

- max berth anchs

```
def max_anch_rule(model,j):
    return sum(model.shipAssignment[i,j] for i in model.Ships) <=
model.berthNumberOfAnchs[j]
model.max_anch_limit = Constraint(model.Berths, rule=max_anch_rule)

def min_anch_rule(model):
    return sum(model.shipAssignment[i,'B3'] for i in model.Ships) >=
model.berthNumberOfAnchs['B3']
model.min_anch_limit = Constraint(rule=min_anch_rule)
```

- length of ship and berth

```
def length_of_berth_rule(model,i,j):
    return model.shipAssignment[i,j]*model.shipLength[i] <=
model.berthLength[j]
model.length_of_berth_limit =
Constraint(model.Ships,model.Berths,rule=length_of_berth_rule)
```

- Si>=Ei

```
def unload_after_arrival_rule(model,i):
    return model.shipUnloadStart[i] >= model.shipArrivalTime[i]
model.unload_after_arrival_rule = Constraint(model.Ships,
rule=unload_after_arrival_rule)

#8- Wi = Si-Ei

def wait_time_bottom_rule(model,i):
    return model.shipWaitingTime[i] >= model.shipUnloadStart[i]-
model.shipArrivalTime[i]

model.wait_time_bottom_limit = Constraint(model.Ships, rule =
wait_time_bottom_rule)

def wait_time_top_rule(model,i):
    return model.shipWaitingTime[i] <= model.shipUnloadStart[i]-
model.shipArrivalTime[i]

model.wait_time_top_limit = Constraint(model.Ships, rule =
wait_time_top_rule)
```

- The `Objective` component is used to define the cost objective. This component uses a rule function to construct the objective expression: $sense=minimize$ means we want to minimize the cost.

```
def cost_rule(model):
    return sum(model.berthIncreaseCapCost[j]*model.berthCapIncrease[j]
for j in model.Berths)+\
           sum(model.shipCostOfWaiting[i] for i in model.Ships)+\
```

```
5*sum(sum(model.shipAssignment[i,j]*model.unloadShipInAnchTime[i,j]
for j in model.Berths)\
                for i in model.Ships)
model.cost = Objective(rule=cost_rule, sense=minimize)
```

# model data

since we have made an abstract model, we can add the data after creating model, to see the data we are feeding the model

execute command below or have a look at parameters.dat:

```
!cat parameters.dat

param:  Ships: shipCargoSize  shipLength shipPriority
shipArrivalTime:=
        S1      6               114         0.963796539422219
4.76552698703798
        S2      18              126         0.773138121490224
1.00459250301696
        S3      20              115         0.441550822305409
3.65299865044074
        S4      5               102         0.981939254756643
6.05417812493725
        S5      11              102         0.190926111844882
1.86918947987366
        S6      19              131         0.348681095490094
2.11799021925453
        S7      20              119         0.970486629114157
4.62295182452406
        S8      13              138         0.445607269149139
6.60117838343523 ;


param:  Berths: berthNumberOfAnchs berthCapacity berthLength berthLag
berthIncreaseCapCost berthMaxCapacityIncrease:=
        B1       2                  57            118
6.92209432152046   5.87153988992838     29
        B2       2                  40            236
9.72782535009737   5.88044719696542     15
        B3       3                  52            120
9.56481291358562   8.86424673153353     49
        B4       2                  30            138
5.74830826792552   6.64031965414847     17 ;

param shipPriorityMatrix :
            S1 S2 S3 S4 S5 S6 S7 S8 :=
        S1   0 1 1 0 1 1 0 1
        S2   0 0 1 0 1 1 0 1
```

```
        S3   0 0 0 0 1 1 0 0
        S4   1 1 1 0 1 1 1 1
        S5   0 0 0 0 0 0 0 0
        S6   0 0 0 0 1 0 0 0
        S7   1 1 1 0 1 1 0 1
        S8   0 0 1 0 1 1 0 0 ;

param unloadShipInAnchTime :
           B1          B2          B3          B4         :=
        S1 9.306945068 8.320459113 9.041338719 8.655545623
        S2 8.499993353 9.905633818 9.993113985 8.089112765
        S3 9.720322075 9.206381222 8.763211972 8.567236436
        S4 9.349929694 8.913662302 9.371722971 9.32369264
        S5 8.265956289 9.535675628 9.964826498 9.938776321
        S6 9.226653641 8.088521266 8.008110288 8.267945054
        S7 9.882004543 8.605721124 8.732291203 9.796392489
        S8 8.62872761  9.097964368 8.872061915 8.129988352  ;
```

# Solution

To get the result of a problem,you can execute the command below: (plots are not shown using this command)

make sure to replace `-problem-number` with correct number.

Use:

```
-a: to solve main problem

-b: to solve the relaxation of Berth3 constraint

-c: sensitivity analysis 1

-d: sensitivity analysis 2

-e: sensitivity analysis 3

-f: sensitivity analysis 4

-g: sensitivity analysis 5

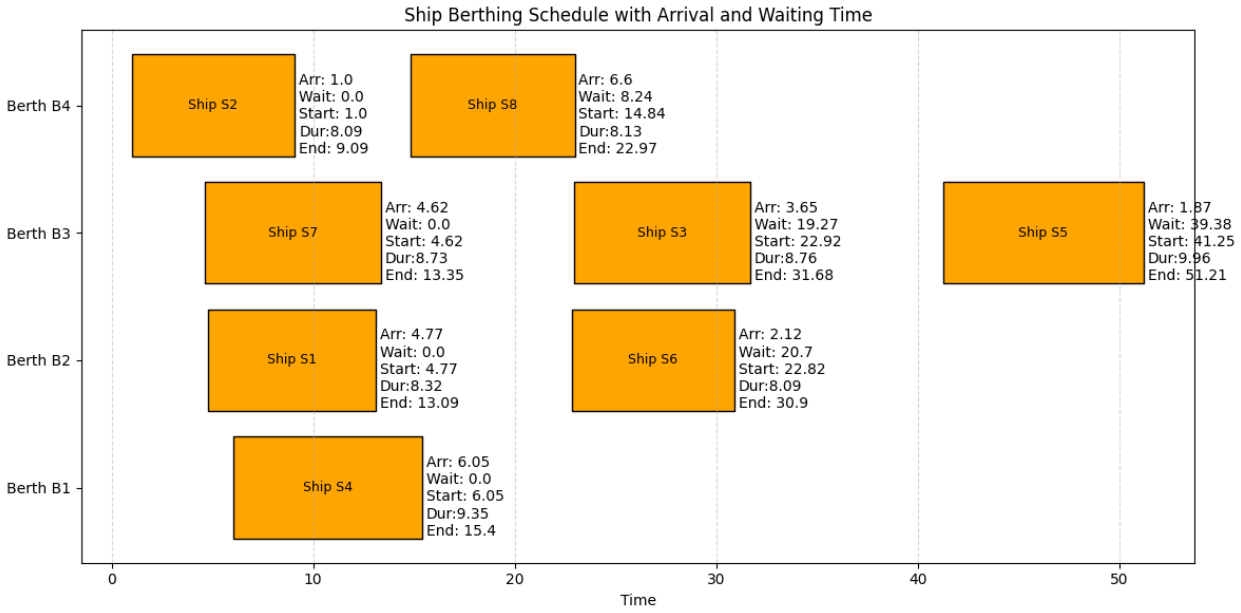
%matplotlib inline
%run model_runner.py -problem-number
```

## Problem A (main)

Execute code below to see the results:

```
%matplotlib inline
%run model_runner.py -a
```

```
total cost: 1803.2706014754367
Ship Berth  Arrival  Start  Wait  Duration   End
  S4    B1     6.05   6.05  0.00      9.35 15.40
  S1    B2     4.77   4.77  0.00      8.32 13.09
  S6    B2     2.12  22.81 20.70      8.09 30.90
  S7    B3     4.62   4.62  0.00      8.73 13.36
  S3    B3     3.65  22.92 19.27      8.76 31.68
  S5    B3     1.87  41.25 39.38      9.96 51.21
  S2    B4     1.00   1.00  0.00      8.09  9.09
  S8    B4     6.60  14.84  8.24      8.13 22.97
```




```
<Figure size 640x480 with 0 Axes>
```

### Explanation

As we see, the total cost for the main problem is

$$cost = 1803.271$$

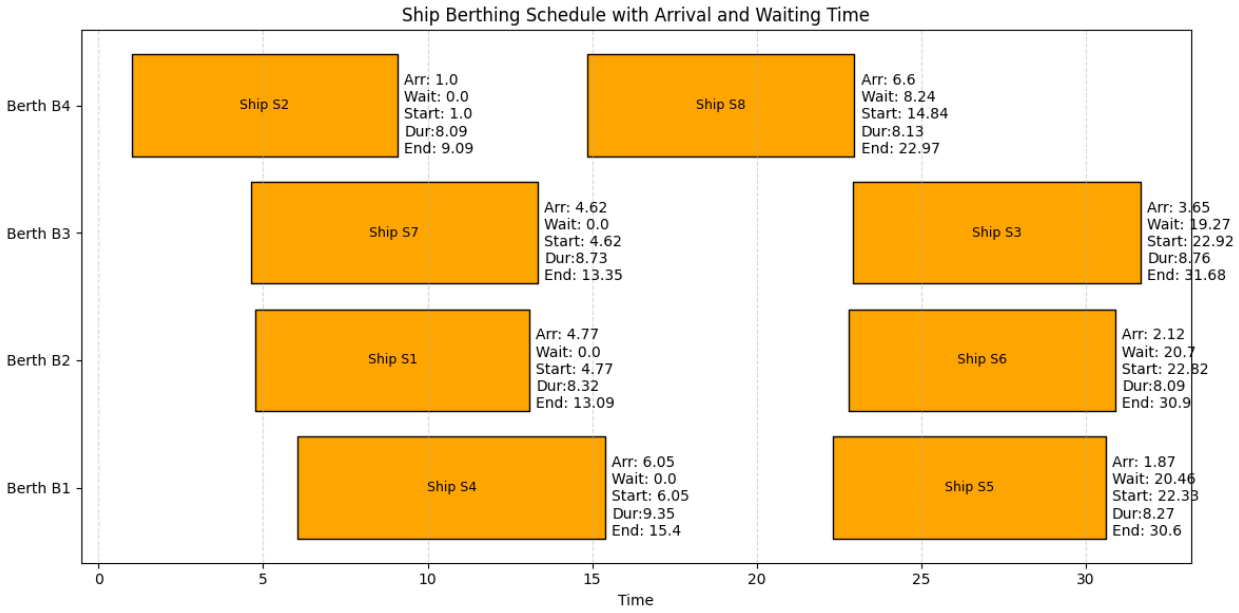
The sequencing of ships in berths is shown in both graphical and text way.

## Problem B (relaxation)

Execute code below to see the resuls:

```
%matplotlib inline
%run model_runner.py -b
```

```
total cost: 1605.5574639445936
Ship Berth  Arrival  Start  Wait  Duration   End
  S4    B1     6.05   6.05  0.00      9.35 15.40
  S5    B1     1.87  22.33 20.46      8.27 30.59
  S1    B2     4.77   4.77  0.00      8.32 13.09
  S6    B2     2.12  22.81 20.70      8.09 30.90
  S7    B3     4.62   4.62  0.00      8.73 13.36
  S3    B3     3.65  22.92 19.27      8.76 31.68
  S2    B4     1.00   1.00  0.00      8.09  9.09
  S8    B4     6.60  14.84  8.24      8.13 22.97
```




```
<Figure size 640x480 with 0 Axes>
```

### Explanation

As we expected, when we relax the berth3 lower limit, because the feasible space gets bigger, the answer could be lower, and it is lower: (ship S5 unloads at berth B1 instead of B3, so it waits less and the cost reduces)

$$cost = 1605.557$$

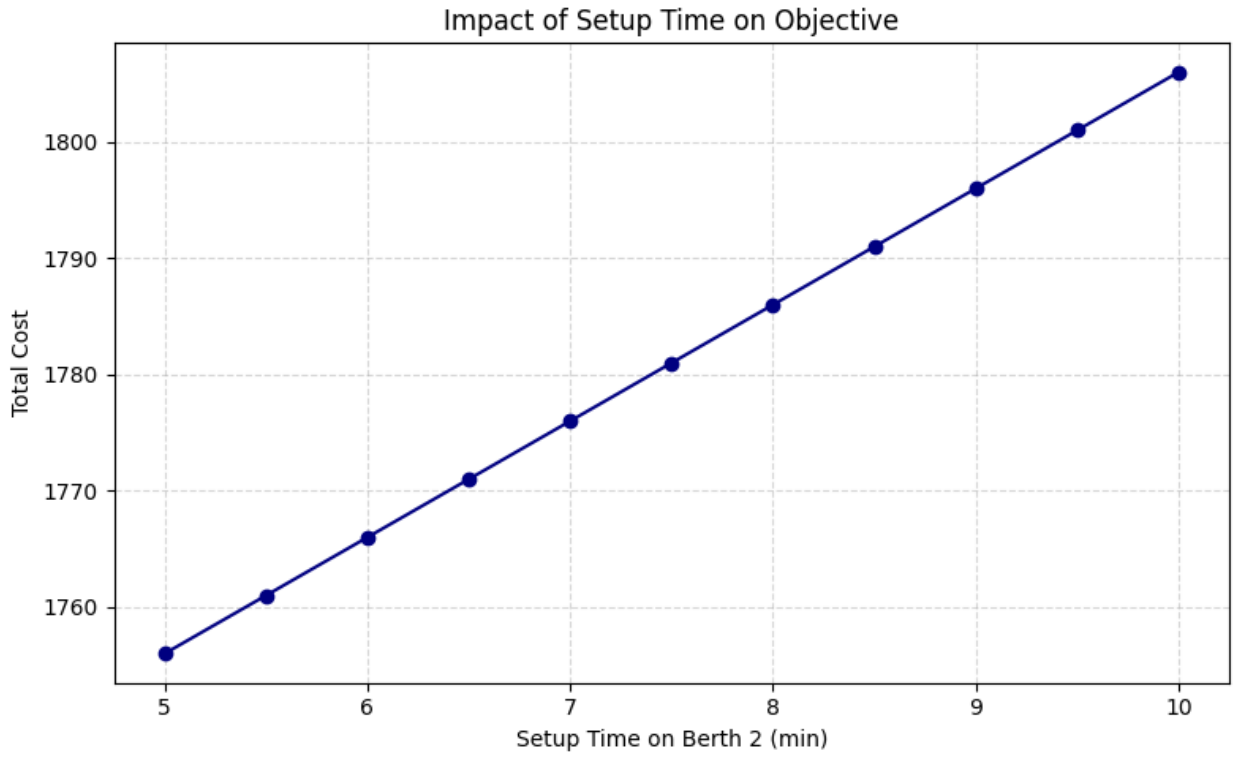
## Problem C (sensitivity analysis 1)

Execute code below to see the resuls:

```
%matplotlib inline
%run model_runner.py -c

Lag B2 = 5.0, Objective = 1755.9923486353946
Lag B2 = 5.5, Objective = 1760.9923486353946
```

```
Lag B2 = 6.0, Objective = 1765.9923486353946
Lag B2 = 6.5, Objective = 1770.9923486353946
Lag B2 = 7.0, Objective = 1775.9923486353946
Lag B2 = 7.5, Objective = 1780.9923486353946
Lag B2 = 8.0, Objective = 1785.9923486353946
Lag B2 = 8.5, Objective = 1790.9923486353946
Lag B2 = 9.0, Objective = 1795.9923486353946
Lag B2 = 9.5, Objective = 1800.9923486353946
Lag B2 = 10.0, Objective = 1805.9923486353946
```




```
<Figure size 640x480 with 0 Axes>
```

### Explanation

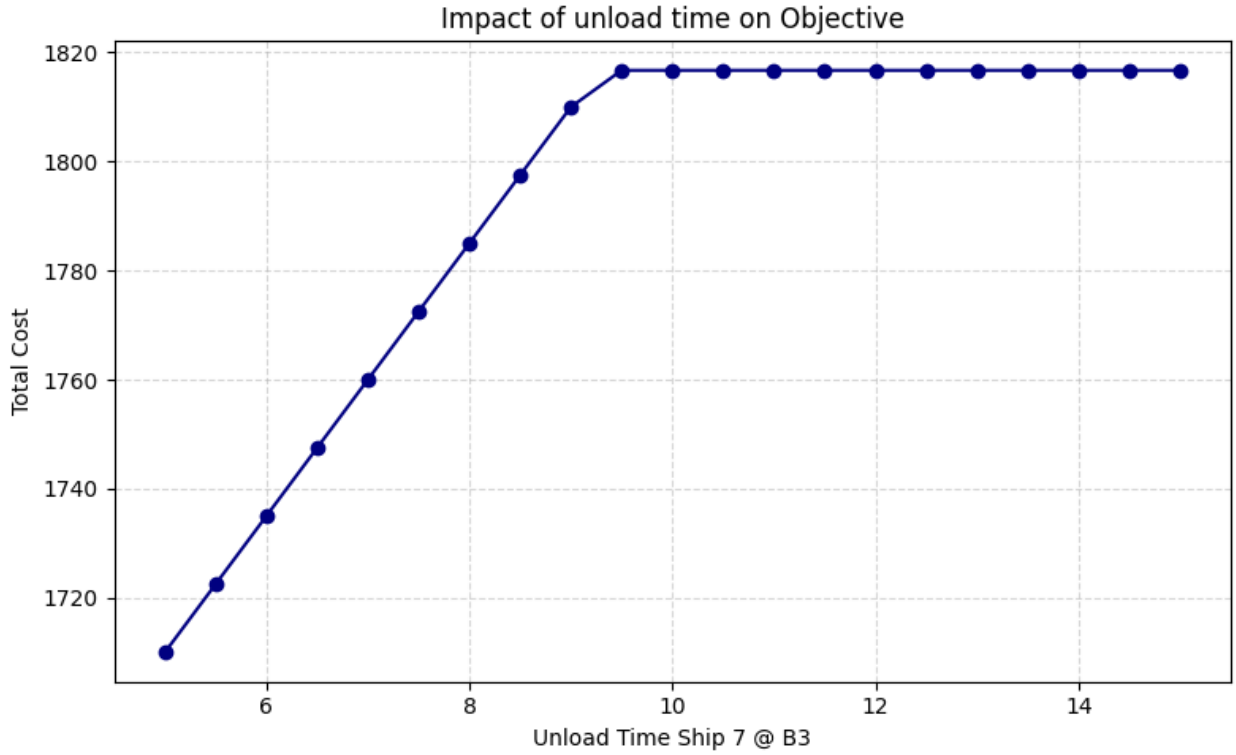
As we know, in berth2 ship S6 unloads as the second ship, so if the berthLag becomes more and more, the ship S6 will have more waiting time and that causes the increase in total cost. if we increase setup time(berthLag) more, the ship S6 might unload in other berths and then the slope of graph will be 0.

## Problem D (sensitivity analysis 2)

Execute code below to see the resuls:

```
%matplotlib inline
%run model_runner.py -d
```

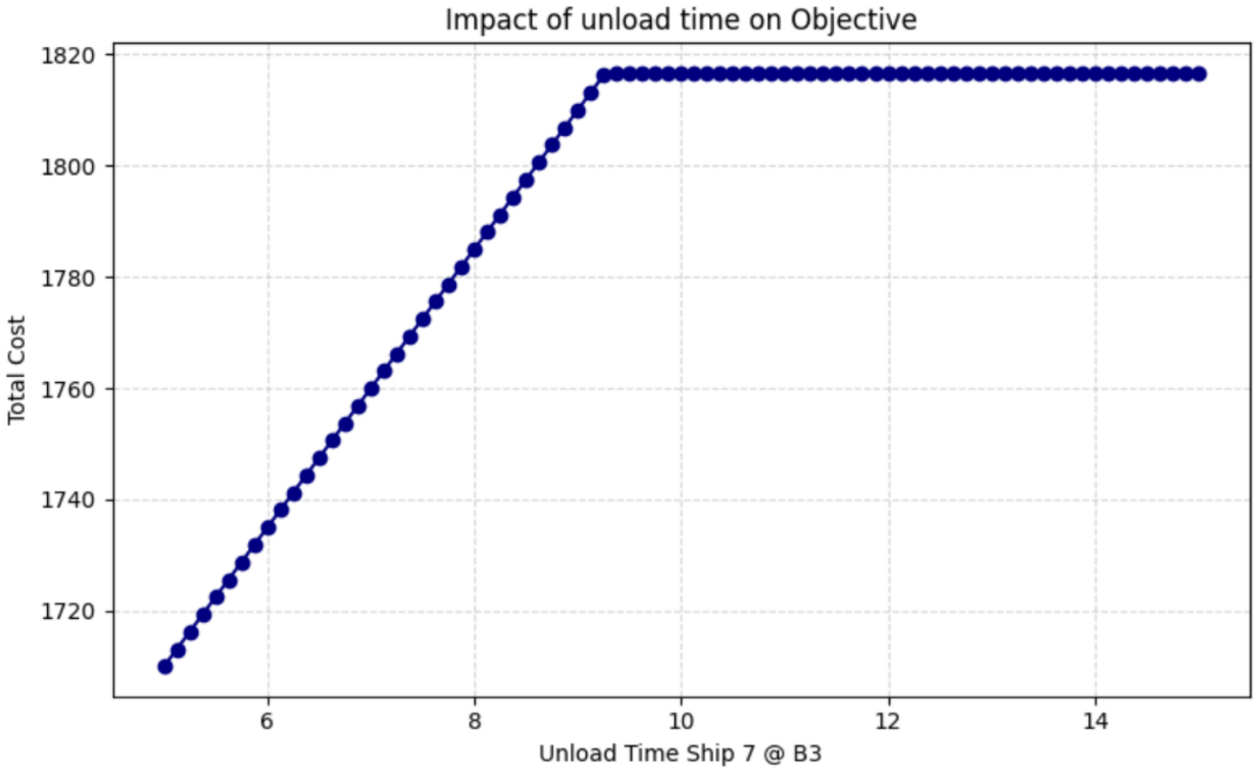
```
Unload Time Ship 7 @ B3 = 5.0, Objective = 1709.9633210280635
Unload Time Ship 7 @ B3 = 5.5, Objective = 1722.4633210280635
Unload Time Ship 7 @ B3 = 6.0, Objective = 1734.9633210280635
Unload Time Ship 7 @ B3 = 6.5, Objective = 1747.4633210280635
Unload Time Ship 7 @ B3 = 7.0, Objective = 1759.9633210280635
Unload Time Ship 7 @ B3 = 7.5, Objective = 1772.4633210280635
Unload Time Ship 7 @ B3 = 8.0, Objective = 1784.9633210280635
Unload Time Ship 7 @ B3 = 8.5, Objective = 1797.4633210280635
Unload Time Ship 7 @ B3 = 9.0, Objective = 1809.9633210280635
Unload Time Ship 7 @ B3 = 9.5, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 10.0, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 10.5, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 11.0, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 11.5, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 12.0, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 12.5, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 13.0, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 13.5, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 14.0, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 14.5, Objective = 1816.7014710251874
Unload Time Ship 7 @ B3 = 15.0, Objective = 1816.7014710251874
```




```
<Figure size 640x480 with 0 Axes>
```

### Explanation

Because the ship S7 unloads as the first ship, the increase in unload time makes the other ships waiting time more, so the cost increases, but with a high increase in the unload time, the ship S7 unloads as second(maybe) and third ship(here the slope becomes 0), we make another graph with higher resolution to see that if the ship S7 unloads as the second ship or not.




We can't say that ship S7 unloads as the second ship, because the slope suddenly becomes 0 and that means the S7 in unloading as the last ship.

## Problem E (SEnsitivity analysis 3)

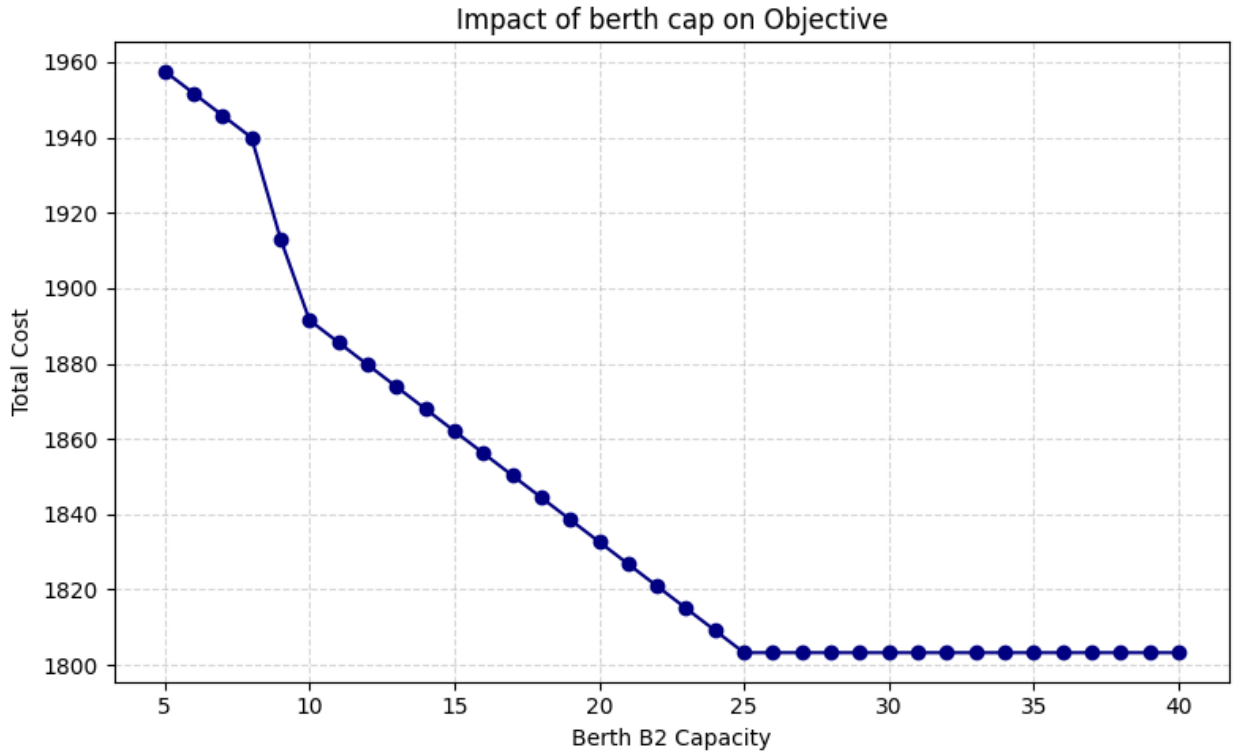
Execute code below to see the resuls:

```
%matplotlib inline
%run model_runner.py -e

Berth B2 Capacity = 5, cap inc: 14.0, Objective = 1957.56325148999
Berth B2 Capacity = 6, cap inc: 13.0, Objective = 1951.6828042930247
total cost: 1951.6828042930247
Ship Berth  Arrival  Start  Wait  Duration   End
  S4    B1     6.05   6.05  0.00      9.35 15.40
  S1    B2     4.77   4.77  0.00      8.32 13.09
  S8    B2     6.60  22.81 16.21      9.10 31.91
  S7    B3     4.62   4.62  0.00      8.73 13.36
  S3    B3     3.65  22.92 19.27      8.76 31.68
  S5    B3     1.87  41.25 39.38      9.96 51.21
```

```
  S2    B4     1.00   1.00  0.00      8.09  9.09
  S6    B4     2.12  14.84 12.72      8.27 23.11
Berth B2 Capacity = 7, cap inc: 12.0, Objective = 1945.8023570960593
Berth B2 Capacity = 8, cap inc: 11.0, Objective = 1939.9219098990939
Berth B2 Capacity = 9, cap inc: 15.0, Objective = 1913.0469454937986
total cost: 1913.0469454937986
Ship Berth  Arrival  Start  Wait  Duration   End
  S1    B1     4.77   4.77  0.00      9.31 14.07
  S4    B2     6.05   6.05  0.00      8.91 14.97
  S6    B2     2.12  24.70 22.58      8.09 32.78
  S7    B3     4.62   4.62  0.00      8.73 13.36
  S3    B3     3.65  22.92 19.27      8.76 31.68
  S5    B3     1.87  41.25 39.38      9.96 51.21
  S2    B4     1.00   1.00  0.00      8.09  9.09
  S8    B4     6.60  14.84  8.24      8.13 22.97
Berth B2 Capacity = 10, cap inc: 15.0, Objective = 1891.477309429918
Berth B2 Capacity = 11, cap inc: 14.0, Objective = 1885.5968622329524
Berth B2 Capacity = 12, cap inc: 13.0, Objective = 1879.7164150359868
Berth B2 Capacity = 13, cap inc: 12.0, Objective = 1873.8359678390216
Berth B2 Capacity = 14, cap inc: 11.0, Objective = 1867.955520642056
Berth B2 Capacity = 15, cap inc: 10.0, Objective = 1862.0750734450908
total cost: 1862.0750734450908
Ship Berth  Arrival  Start  Wait  Duration   End
  S4    B1     6.05   6.05  0.00      9.35 15.40
  S1    B2     4.77   4.77  0.00      8.32 13.09
  S6    B2     2.12  22.81 20.70      8.09 30.90
  S7    B3     4.62   4.62  0.00      8.73 13.36
  S3    B3     3.65  22.92 19.27      8.76 31.68
  S5    B3     1.87  41.25 39.38      9.96 51.21
  S2    B4     1.00   1.00  0.00      8.09  9.09
  S8    B4     6.60  14.84  8.24      8.13 22.97
Berth B2 Capacity = 16, cap inc: 9.0, Objective = 1856.1946262481251
Berth B2 Capacity = 17, cap inc: 8.0, Objective = 1850.31417905116
Berth B2 Capacity = 18, cap inc: 7.0, Objective = 1844.4337318541943
Berth B2 Capacity = 19, cap inc: 6.0, Objective = 1838.5532846572291
Berth B2 Capacity = 20, cap inc: 5.0, Objective = 1832.6728374602635
Berth B2 Capacity = 21, cap inc: 4.0, Objective = 1826.7923902632983
Berth B2 Capacity = 22, cap inc: 3.0, Objective = 1820.9119430663327
Berth B2 Capacity = 23, cap inc: 2.0, Objective = 1815.0314958693675
Berth B2 Capacity = 24, cap inc: 1.0, Objective = 1809.1510486724019
Berth B2 Capacity = 25, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 26, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 27, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 28, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 29, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 30, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 31, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 32, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 33, cap inc: 0.0, Objective = 1803.2706014754367
```

```
Berth B2 Capacity = 34, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 35, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 36, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 37, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 38, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 39, cap inc: 0.0, Objective = 1803.2706014754367
Berth B2 Capacity = 40, cap inc: 0.0, Objective = 1803.2706014754367
```




```
<Figure size 640x480 with 0 Axes>
```

### Explanation

We see that as the beginning capacity increases, the additional capacity decreases and therefore the total cost decreases, but why the graph has three peaces? because at first, in range 5 to 8, the solution sends Ships S1 and S8 to Berth B2, that means in that range we need the capacity to be 19 at least. In berth capacity of 9, the solution sends Ships B4 and B6 to the Berth B2, that means we need the capacity to be at least 24, and for the beginning capacities more than 10, the solution sends Ships S1 and S6 to the Berth B2, so the minimum capacity should be 25, for more than 25 we need no extra capacity and the cost does not change.

That is why we have a three peace graph, because the assignment of ships changes, a thing that does not happen in next problem.

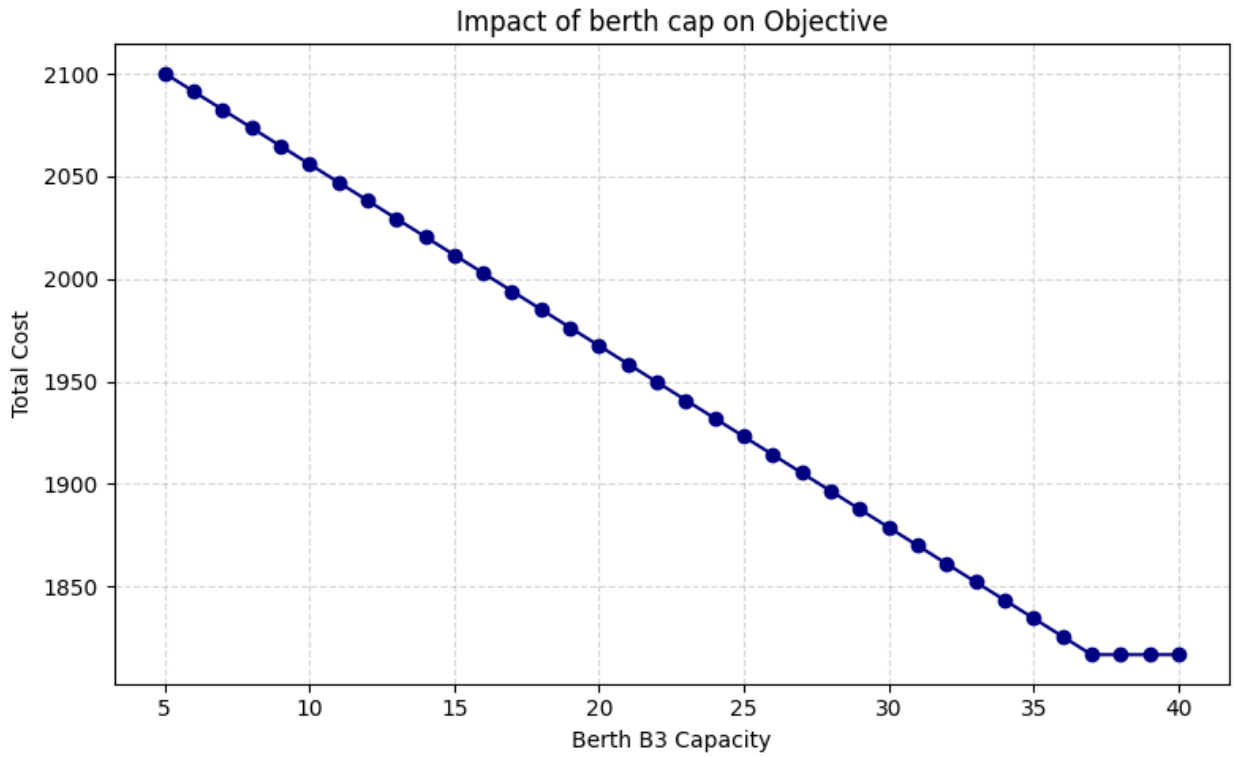
## Problem F(sensitivity analysis 4)

Execute code below to see the results:

```
%matplotlib inline
%run model_runner.py -f
```

```
Berth B3 Capacity = 5, cap inc: 32.0, Objective = 2100.3573664342603
Berth B3 Capacity = 6, cap inc: 31.0, Objective = 2091.4931197027267
Berth B3 Capacity = 7, cap inc: 30.0, Objective = 2082.6288729711932
Berth B3 Capacity = 8, cap inc: 29.0, Objective = 2073.7646262396597
Berth B3 Capacity = 9, cap inc: 28.0, Objective = 2064.900379508126
Berth B3 Capacity = 10, cap inc: 27.0, Objective = 2056.0361327765927
Berth B3 Capacity = 11, cap inc: 26.0, Objective = 2047.1718860450592
Berth B3 Capacity = 12, cap inc: 25.0, Objective = 2038.3076393135257
Berth B3 Capacity = 13, cap inc: 24.0, Objective = 2029.4433925819922
Berth B3 Capacity = 14, cap inc: 23.0, Objective = 2020.5791458504586
Berth B3 Capacity = 15, cap inc: 22.0, Objective = 2011.7148991189251
Berth B3 Capacity = 16, cap inc: 21.0, Objective = 2002.8506523873916
Berth B3 Capacity = 17, cap inc: 20.0, Objective = 1993.986405655858
Berth B3 Capacity = 18, cap inc: 19.0, Objective = 1985.1221589243246
Berth B3 Capacity = 19, cap inc: 18.0, Objective = 1976.257912192791
Berth B3 Capacity = 20, cap inc: 17.0, Objective = 1967.3936654612576
Berth B3 Capacity = 21, cap inc: 16.0, Objective = 1958.529418729724
Berth B3 Capacity = 22, cap inc: 15.0, Objective = 1949.6651719981905
Berth B3 Capacity = 23, cap inc: 14.0, Objective = 1940.800925266657
Berth B3 Capacity = 24, cap inc: 13.0, Objective = 1931.9366785351235
Berth B3 Capacity = 25, cap inc: 12.0, Objective = 1923.07243180359
Berth B3 Capacity = 26, cap inc: 11.0, Objective = 1914.2081850720565
Berth B3 Capacity = 27, cap inc: 10.0, Objective = 1905.343938340523
Berth B3 Capacity = 28, cap inc: 9.0, Objective = 1896.4796916089895
Berth B3 Capacity = 29, cap inc: 8.0, Objective = 1887.6154448774555
Berth B3 Capacity = 30, cap inc: 7.0, Objective = 1878.751198145922
Berth B3 Capacity = 31, cap inc: 6.0, Objective = 1869.8869514143885
Berth B3 Capacity = 32, cap inc: 5.0, Objective = 1861.022704682855
Berth B3 Capacity = 33, cap inc: 4.0, Objective = 1852.1584579513215
Berth B3 Capacity = 34, cap inc: 3.0, Objective = 1843.294211219788
Berth B3 Capacity = 35, cap inc: 2.0, Objective = 1834.4299644882544
Berth B3 Capacity = 36, cap inc: 1.0, Objective = 1825.565717756721
Berth B3 Capacity = 37, cap inc: 0.0, Objective = 1816.7014710251874
Berth B3 Capacity = 38, cap inc: 0.0, Objective = 1816.7014710251874
Berth B3 Capacity = 39, cap inc: 0.0, Objective = 1816.7014710251874
Berth B3 Capacity = 40, cap inc: 0.0, Objective = 1816.7014710251874
total cost: 1816.7014710251874
```

| Ship | Berth | Arrival | Start | Wait | Duration | End |
|---|---|---|---|---|---|---|
| S4 | B1 | 6.05 | 6.05 | 0.00 | 9.35 | 15.40 |
| S7 | B2 | 4.62 | 4.62 | 0.00 | 8.61 | 13.23 |
| S6 | B2 | 2.12 | 22.96 | 20.84 | 8.09 | 31.05 |
| S1 | B3 | 4.77 | 4.77 | 0.00 | 9.04 | 13.81 |
| S3 | B3 | 3.65 | 23.37 | 19.72 | 8.76 | 32.13 |
| S5 | B3 | 1.87 | 41.70 | 39.83 | 9.96 | 51.66 |
| S2 | B4 | 1.00 | 1.00 | 0.00 | 8.09 | 9.09 |
| S8 | B4 | 6.60 | 14.84 | 8.24 | 8.13 | 22.97 |




```
<Figure size 640x480 with 0 Axes>
```

### Explanation

In the range 5 to 40 for Berth B3 capacity, the solution sends Ships S1,S3 and S5 to the Berth B3, it means berth needs a capacity of at least 37 which is what happens here, we increase capacity to 37, and more capacity we have in the beginning, the less increase we need and that means less cost.

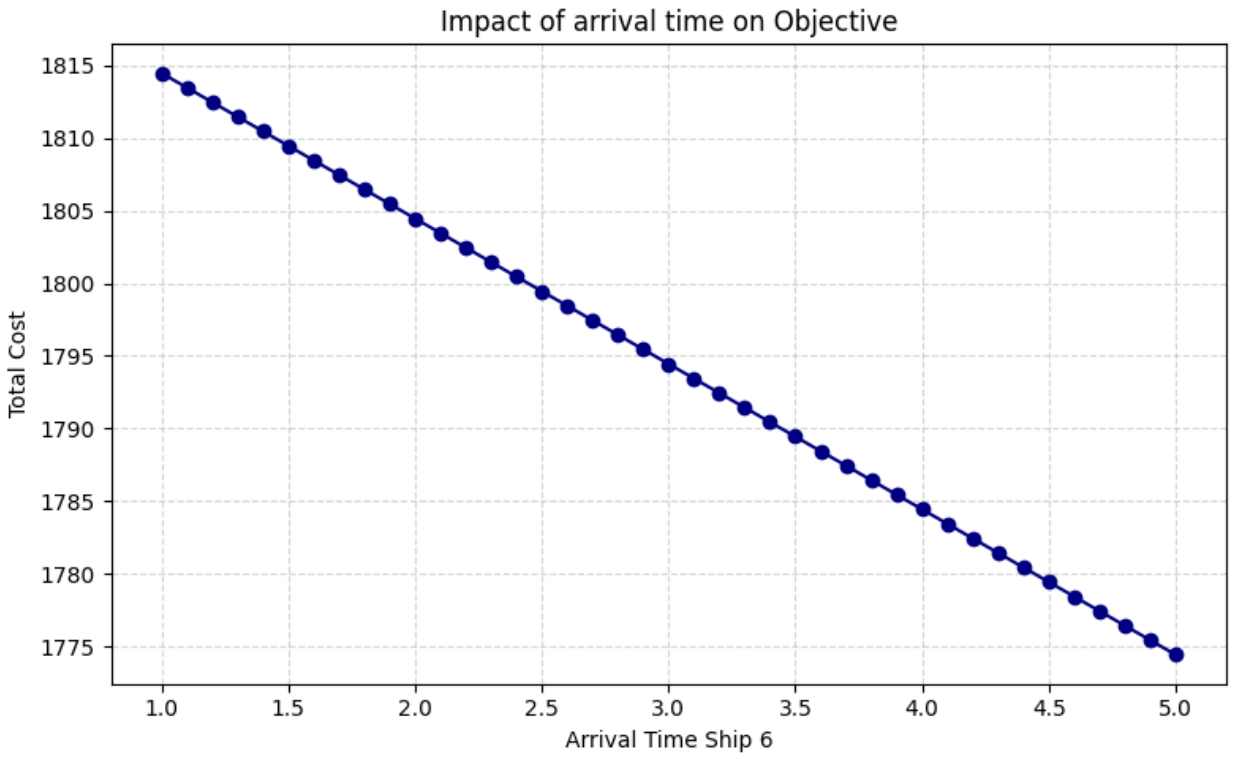
## Problem G (sensitivity analysis 5)

Execute code below to see the resuls:

```
%matplotlib inline
%run model_runner.py -g

Arrival Time Ship 6 = 1.0, Objective = 1814.4505036679816
Arrival Time Ship 6 = 1.1, Objective = 1813.4505036679816
Arrival Time Ship 6 = 1.2, Objective = 1812.4505036679816
Arrival Time Ship 6 = 1.3, Objective = 1811.4505036679816
Arrival Time Ship 6 = 1.4, Objective = 1810.4505036679816
Arrival Time Ship 6 = 1.5, Objective = 1809.4505036679816
Arrival Time Ship 6 = 1.6, Objective = 1808.4505036679816
Arrival Time Ship 6 = 1.7, Objective = 1807.4505036679816
Arrival Time Ship 6 = 1.8, Objective = 1806.4505036679816
Arrival Time Ship 6 = 1.9, Objective = 1805.4505036679816
```

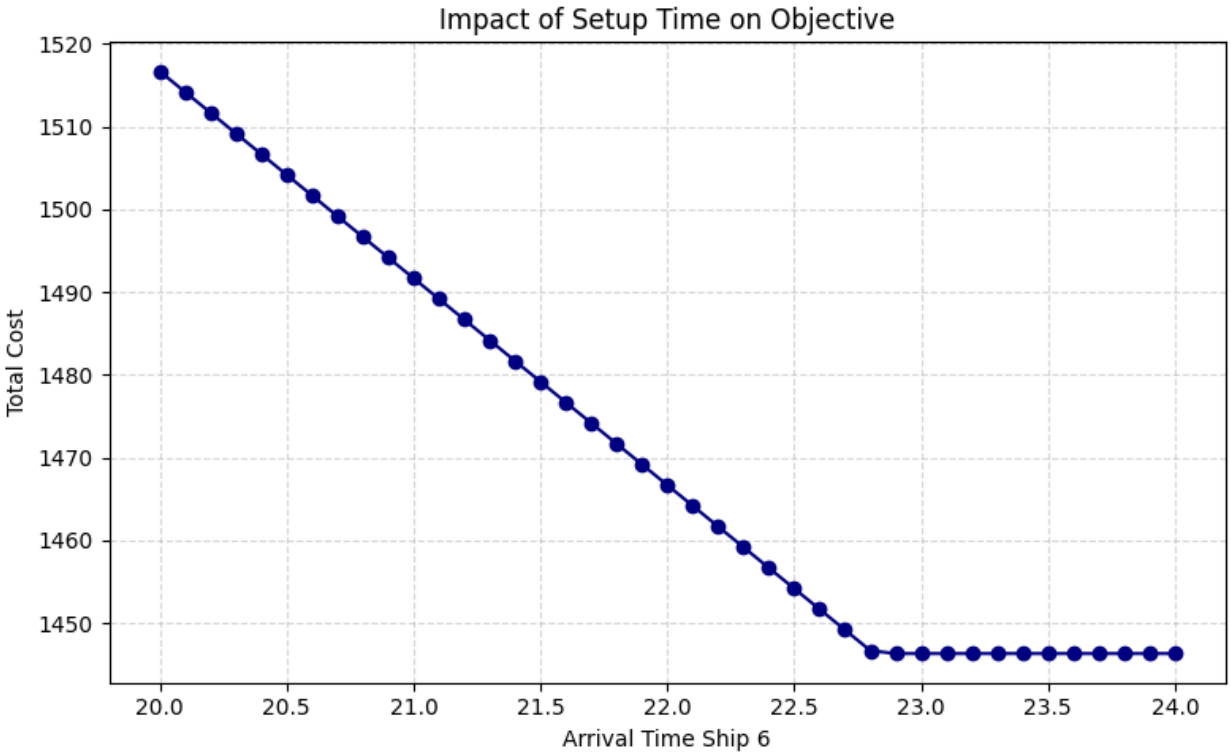
```
Arrival Time Ship 6 = 2.0, Objective = 1804.4505036679816
Arrival Time Ship 6 = 2.1, Objective = 1803.4505036679816
Arrival Time Ship 6 = 2.2, Objective = 1802.4505036679816
Arrival Time Ship 6 = 2.3, Objective = 1801.4505036679816
Arrival Time Ship 6 = 2.4, Objective = 1800.4505036679816
Arrival Time Ship 6 = 2.5, Objective = 1799.4505036679816
Arrival Time Ship 6 = 2.6, Objective = 1798.4505036679816
Arrival Time Ship 6 = 2.7, Objective = 1797.4505036679816
Arrival Time Ship 6 = 2.8, Objective = 1796.4505036679816
Arrival Time Ship 6 = 2.9, Objective = 1795.4505036679816
Arrival Time Ship 6 = 3.0, Objective = 1794.4505036679816
Arrival Time Ship 6 = 3.1, Objective = 1793.4505036679816
Arrival Time Ship 6 = 3.2, Objective = 1792.4505036679816
Arrival Time Ship 6 = 3.3, Objective = 1791.4505036679816
Arrival Time Ship 6 = 3.4, Objective = 1790.4505036679816
Arrival Time Ship 6 = 3.5, Objective = 1789.4505036679816
Arrival Time Ship 6 = 3.6, Objective = 1788.4505036679816
Arrival Time Ship 6 = 3.7, Objective = 1787.4505036679816
Arrival Time Ship 6 = 3.8, Objective = 1786.4505036679816
Arrival Time Ship 6 = 3.9, Objective = 1785.4505036679816
Arrival Time Ship 6 = 4.0, Objective = 1784.4505036679816
Arrival Time Ship 6 = 4.1, Objective = 1783.4505036679816
Arrival Time Ship 6 = 4.2, Objective = 1782.4505036679816
Arrival Time Ship 6 = 4.3, Objective = 1781.4505036679816
Arrival Time Ship 6 = 4.4, Objective = 1780.4505036679816
Arrival Time Ship 6 = 4.5, Objective = 1779.4505036679816
Arrival Time Ship 6 = 4.6, Objective = 1778.4505036679816
Arrival Time Ship 6 = 4.7, Objective = 1777.4505036679816
Arrival Time Ship 6 = 4.8, Objective = 1776.4505036679816
Arrival Time Ship 6 = 4.9, Objective = 1775.4505036679816
Arrival Time Ship 6 = 5.0, Objective = 1774.4505036679816
```




```
<Figure size 640x480 with 0 Axes>
```

## Explanation

As we know, the ship S6 arrives in 2.12 and unloads in 22.82, so we know that for arrival times between 2.12 and 22.82, the ship will unload after previous ship as it was before and the cost will reduce by cost of waiting multiplied by latency in arrival, which is happening as we see, for arrival times before 2.12, which means arrival times between 1 and 2.12, we see that ship S6 is still unloading as second ship so the graph will have the same slope, but for arrival times after 22.82, because the waiting time would be 0, we will see that slope of cost will be 0, we will plot that too.


Impact of Setup Time on Objective
Total Cost
Arrival Time Ship 6
1520
1510
1500
1490
1480
1470
1460
1450
20.0
20.5
21.0
21.5
22.0
22.5
23.0
23.5
24.0